Calculator DevOps

How to run the project and test it

CSE 816 Mini-Project · Spring Boot 3.3 · Java 21 · Maven · Docker · Docker Compose · Jenkins

This guide shows how to run the project and check that every part works, from a quick test on your laptop to the full Jenkins pipeline. Each step gives the commands to type and what you should see. Run every command from the project folder, `calculator-devops`.

Quick check (Java and Maven only)

```
cd calculator-devops
mvn test
mvn spring-boot:run
```

`mvn test` should end with **Tests run: 16, Failures: 0** and **BUILD SUCCESS**. `mvn spring-boot:run` keeps running; in a second terminal run `curl 'http://localhost:8080/api/v1/add?a=2&b=3'` and you should get `{"operation":"add","a":2.0,"b":3.0,"result":5.0}`.

What each step needs

Step	What you do	You need
1	Check the tools and find your way around the project	—
2	Run the automated tests	Java 21, Maven
3	Run the app and test the API by hand	Java 21, Maven, curl
4	Run the app in a Docker container	Docker (Colima or Docker Desktop)
5	Run the app behind Nginx with Docker Compose	Docker (Colima or Docker Desktop)
6	Set up, run and test the CI/CD pipeline	Jenkins, the GitHub repo, a Docker Hub account
7	Check everything before the demo	All of the above
8–9	Troubleshooting and the final test checklist	—

1. Before you start

Tool	Version	How to check
Java (JDK)	21	`java -version` shows 21
Maven	3.9 or newer	`mvn -version` shows Maven 3.9+ and Java version 21
Docker: Colima or Docker Desktop	Compose v2.17 or newer	`docker --version` and `docker compose version`
Jenkins	LTS	Opens in the browser (set up in section 6)
curl	any	`curl --version`

Mac with Homebrew: if `mvn -version` shows Java 17, the build stops with **release version 21 not supported**. Install JDK 21 with `brew install openjdk@21` and run `export JAVA_HOME=/opt/homebrew/opt/openjdk@21` in each new terminal, or add that line to `~/.zshrc`.

Docker on this Mac runs in Colima, a command-line Docker. After a reboot, start it with `colima start` before using any `docker` command. With Docker Desktop instead, just open the app.

Accounts. The project uses GitHub `Mayanktan`, Docker Hub `mayanktan`, and sends build emails to `mayanktanwar01234@gmail.com`. To use other accounts, change them in `Jenkinsfile`, `docker-compose.yml` and `scripts/preflight.sh`.

Where things are

Path	What it is
`src/main/java/com/example/calculator/`	The REST API: controller, service, DTOs and error handling
`src/test/java/com/example/calculator/`	`CalculatorServiceTest` (unit tests) and `CalculatorControllerIT` (integration tests)
`pom.xml`	Maven build: Spring Boot 3.3.4, Java 21 and the test setup
`Dockerfile`, `.dockerignore`	Two-stage image build: Maven and JDK first, then a small JRE image
`docker-compose.yml`, `nginx/nginx.conf`	The app plus an Nginx reverse proxy on a shared network
`Jenkinsfile`	The CI/CD pipeline: six stages plus the email
`scripts/preflight.sh`	Checks to run before a demo

2. Run the automated tests

The project has 16 JUnit 5 tests. Run them all:

```
mvn test
```

OUTPUT (SHORTENED)

```
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0 ... in ...CalculatorServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0 ... in ...CalculatorControllerIT
[INFO] Results:
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Test class	Tests	What it checks
`CalculatorServiceTest`	8 unit tests. Plain JUnit 5 with no Spring, so they run in milliseconds.	add (normal and negative numbers), subtract, multiply (normal and by zero), divide, power, and that dividing by zero throws `DivideByZeroException`
`CalculatorControllerIT`	8 integration tests. They start Spring and call the real endpoints through MockMvc.	add, subtract, multiply and divide return 200 with the right JSON; divide by zero, a missing parameter and a non-number return 400 with an error JSON; `/actuator/health` is UP

Run only some tests

```
mvn test -Dtest=CalculatorServiceTest
mvn test -Dtest='CalculatorControllerIT#divideByZeroReturns400'
```

The first line runs one test class, the second runs one test method. Quoting the second one is a safe habit in zsh, where `#` can have a special meaning.

Test reports and the JAR

- Each run writes JUnit XML reports to `target/surefire-reports/`, one file per test class. Jenkins reads these files to show the test results.
- `mvn clean package` runs the tests and then builds the runnable JAR `target/calculator-devops-1.0.0.jar`. Add `-DskipTests` to build it without running the tests.

3. Run the app and test the API by hand

Start the app. It keeps running in this terminal until you press Ctrl+C.

```
mvn spring-boot:run
```

Or build the JAR and run it, the same way the Docker image does:

```
mvn clean package
java -jar target/calculator-devops-1.0.0.jar
```

Wait for the line `Started CalculatorApplication`. Then, in a second terminal, call each URL path from the table below like this, changing the path each time. The `-i` option also prints the HTTP status line, so you can check the status code.

```
curl -i 'http://localhost:8080/api/v1/add?a=2&b=3'
```

#	URL path	Status	Expected result
1	`/api/v1/add?a=2&b=3`	200	result 5.0
2	`/api/v1/subtract?a=5&b=2`	200	result 3.0
3	`/api/v1/multiply?a=4&b=3`	200	result 12.0
4	`/api/v1/divide?a=10&b=4`	200	result 2.5
5	`/api/v1/power?a=2&b=10`	200	result 1024.0
6	`/api/v1/divide?a=10&b=0`	400	message "Cannot divide by zero"
7	`/api/v1/add?a=2`	400	message "Missing required parameter: b"
8	`/api/v1/add?a=foo&b=3`	400	message "Invalid value for parameter 'a': must be a number"
9	`/api/v1/sqrt?a=4`	404	Not Found: there is no such endpoint
10	`/actuator/health`	200	`"status":"UP"` followed by details

The full responses for rows 1 and 6 look like this. The error body is one line; it is wrapped here to fit the page.

OUTPUT

```
HTTP/1.1 200
Content-Type: application/json
...
{"operation":"add","a":2.0,"b":3.0,"result":5.0}

HTTP/1.1 400
Content-Type: application/json
...
{"timestamp":"2026-10-28T16:27:46.714560Z","status":400,"error":"Bad Request",
 "message":"Cannot divide by zero","path":"/api/v1/divide"}
```

zsh tip: always put the URL in single quotes. Without them, zsh treats ? and & as special characters and you get **no matches found** or a command that runs in the background.

4. Run it in a Docker container

Start Docker first (`colima start`, or open Docker Desktop). Then build the image and start one container:

```
docker build -t mayanktan/calculator-devops:1.0.0 .
docker run --rm -d -p 8080:8080 --name calc mayanktan/calculator-devops:1.0.0
```

Wait about 10 seconds for Spring Boot to start. Then test it, check which user it runs as, and stop it:

```
curl 'http://localhost:8080/api/v1/add?a=2&b=3'
docker exec calc id
docker stop calc
```

- The first `docker build` downloads the Maven and Java base images and about 150 MB of Maven dependencies, so it takes a few minutes. Later builds reuse the cached dependency layer and are much faster, unless `pom.xml` changes.
- `curl` returns the same JSON as in section 3.
- `docker exec calc id` must show the `app` user with a non-zero uid, for example `uid=100(app)`, never `uid=0(root)`. This proves the container does not run as root.
- Because of `--rm`, `docker stop` also removes the container.

Port 8080 busy? Homebrew's Jenkins also uses port 8080. Use `-p 8090:8080` and call `http://localhost:8090` instead.

5. Run the full stack with Docker Compose

Compose starts two containers on a private network: the calculator and an Nginx reverse proxy in front of it. Only Nginx can be reached from your machine, on port 80.

```
docker compose up -d --build
docker compose ps
```

- `up` prints `Container calculator Healthy` and then `Container calc-nginx Started`. Nginx waits until the app passes its health check.
- In `docker compose ps`, `calculator` is **Up (healthy)** and its PORTS column shows only `8080/tcp`, which is internal. `calc-nginx` is **Up** with `0.0.0.0:80->80/tcp`, which is published.

Test through Nginx. It listens on port 80, so there is no port number in the URL:

```
curl 'http://localhost/api/v1/add?a=10&b=5'
curl 'http://localhost/actuator/health'
curl 'http://localhost/'
```

You should get `{"operation":"add","a":10.0,"b":5.0,"result":15.0}`, then `{"status":"UP",...}`, then the small HTML landing page that starts with `<h1>Calculator DevOps</h1>`.

Useful commands while testing

Command	What it does
`docker compose logs -f calculator`	Follows the Spring Boot log. Press Ctrl+C to stop following.
`docker compose up -d`	Run it again on an unchanged stack: nothing restarts and both containers just show Running. The pipeline relies on this.
`docker compose down`	Stops and removes both containers and the network. The image stays.

6. Run and test the CI/CD pipeline

The pipeline runs on every push to `main`: **Checkout, Build, Test, Docker Build, Push, Deploy**, and then it emails the result. Install Jenkins directly on the machine that runs Docker, not inside a container, because the Deploy stage runs `docker compose` on that machine and checks `http://localhost`.

6.1 One-time setup

1. **Install and start Jenkins.** On a Mac:

   ```
   brew install jenkins-lts
   brew services start jenkins-lts
   cat ~/.jenkins/secrets/initialAdminPassword
   ```

   Open `http://localhost:8080`, paste the password, choose **Install suggested plugins** and create your admin user. On Linux, install the `jenkins` package instead and run `sudo usermod -aG docker jenkins`, then restart Jenkins so that it can use Docker.

2. **Check the plugins.** Under Manage Jenkins > Plugins, make sure these are installed: Pipeline, Git, GitHub, Credentials Binding, JUnit, Email Extension and Timestamper.

3. **Point Jenkins to Java 21, Maven and Docker.** Under Manage Jenkins > System > Global properties, tick **Environment variables** and add:

Name	Value
`JAVA_HOME`	`/opt/homebrew/opt/openjdk@21`
`PATH+EXTRA`	`/opt/homebrew/bin:/usr/local/bin`

   `PATH+EXTRA` adds these folders to the PATH that Jenkins uses: Homebrew's `mvn` is in `/opt/homebrew/bin` and Docker's command-line tool is in `/usr/local/bin`.

4. **Add your Docker Hub login.** On hub.docker.com, create a personal access token with Read & Write access. In Jenkins, go to Manage Jenkins > Credentials > System > Global credentials > Add Credentials and fill in: Kind **Username with password**, Username your Docker Hub username, Password the token, ID `dockerhub-creds`. The Jenkinsfile uses this exact ID.

5. **Set up email.** Under Manage Jenkins > System > Extended E-mail Notification: SMTP server `smtp.gmail.com`, port `465`, tick Use SSL, add credentials with your Gmail address and a Gmail **App**

Password (Gmail rejects your normal password), and set the default content type to HTML.

6. **Put your details in the Jenkinsfile.** At the top of `Jenkinsfile`, set `DOCKERHUB_USER` to your Docker Hub username (lowercase) and `NOTIFY_EMAIL` to your email address. Commit and push.
7. **Create the job.** Click New Item, name it `calculator-devops` and choose Pipeline. In the Pipeline section choose **Pipeline script from SCM**, SCM **Git**, Repository URL `https://github.com/Mayanktan/calculator-devops.git`, branch `*/main` and script path `Jenkinsfile`.
8. **Turn on the automatic trigger,** using one of these:
   - **Webhook (preferred).** Run `ngrok http 8080` and set the https address it prints as the Jenkins URL (Manage Jenkins > System). On GitHub, open the repo's Settings > Webhooks > Add webhook: Payload URL `https://<ngrok-address>/github-webhook/` (keep the slash at the end), content type `application/json`, just the push event. In the job settings, tick **GitHub hook trigger for GITScm polling**.
   - **Poll SCM.** In the job settings, tick **Poll SCM** and enter `H/2 * * * *`, so Jenkins checks GitHub every 2 minutes.

6.2 Run the pipeline

Click **Build Now**. The first run takes a few minutes because Maven and Docker download everything; later runs are faster.

Stage	What it does	It fails if
Checkout	Gets the latest commit from GitHub	the repository URL or branch is wrong
Build	`mvn clean package -DskipTests`	the code does not compile
Test	`mvn test`, then publishes the JUnit XML reports	any test fails
Docker Build	Builds the image, tagged with the build number and `latest`	the Dockerfile or Docker is broken
Push	Logs in with `dockerhub-creds` and pushes both tags	the login or the username is wrong
Deploy	`docker compose up -d --wait`, then checks `/actuator/health` through Nginx	the app is not healthy within about a minute

A good run looks like this:

- All six stages are green, and the build page has a **Test Result** link showing 16 passing tests.
- On hub.docker.com the repository shows the new tag (the build number) and `latest`. Make sure the repository is set to Public.
- `docker compose ps` shows the stack that Jenkins deployed, and `curl 'http://localhost/api/v1/add?a=2&b=3'` works.
- An email arrives: **[Jenkins] SUCCESS: calculator-devops #N**.

6.3 Test that a push deploys by itself

1. In `nginx/nginx.conf`, change the welcome text `<h1>Calculator DevOps</h1>` to `<h1>Calculator DevOps - DEMO</h1>`.
2. Commit and push to `main`:

```
git commit -am "demo: change welcome text"
git push
```

3. A new build starts without you clicking anything: within seconds with the webhook, or within 2 minutes with Poll SCM.
4. When it is green, `curl 'http://localhost/'` shows the new text.

6.4 Test that a failing test stops the pipeline

1. In `src/test/java/com/example/calculator/CalculatorServiceTest.java`, change `assertEquals(5.0, service.add(2, 3), 1e-9);` to expect `6.0`. Commit and push.
2. The Test stage turns red, Docker Build, Push and Deploy are skipped, the running app stays as it was, and you get a **[Jenkins] FAILED** email.
3. Change it back to `5.0` and push again. The next build is green.

7. Check everything before the demo

```
./scripts/preflight.sh
```

With no arguments it checks GitHub user `Mayanktan`, Docker Hub user `mayanktan` and Jenkins at `http://localhost:8080`. To check other values, pass them in that order: `./scripts/preflight.sh <github-user> <dockerhub-user> <jenkins-url>`.

The script checks, in order: Docker is running, Jenkins answers, the ngrok tunnel (only reported, never a failure), the stack is healthy through Nginx, and your GitHub and Docker Hub repositories exist. It stops at the first failed check. When everything is fine, the last line is:

OUTPUT

```
All preflight checks passed — ready to demo.
```

8. Troubleshooting

Problem	Fix
`release version 21 not supported` or `invalid target release: 21`	Maven is using an older JDK. Run `export JAVA_HOME=/opt/homebrew/opt/openjdk@21` and check `mvn -version`. For Jenkins, set `JAVA_HOME` as in step 3 of section 6.1.
`Port 8080 was already in use` when starting the app	Jenkins is probably on 8080. Start the app on another port with `java -jar target/calculator-devops-1.0.0.jar --server.port=8090` and use `http://localhost:8090`.
`docker build` fails with exit code 137	Docker ran out of memory. Give it at least 4 GB: `colima stop` then `colima start --memory 4`, or in Docker Desktop under Settings > Resources.
`Cannot connect to the Docker daemon`	Docker is not running. Start it with `colima start`.
`port is already allocated` for port 80	Another program is using port 80. Find it with `sudo lsof -i :80` and stop it.
`calculator` never becomes healthy, or the Deploy stage fails	Read the app's log with `docker compose logs calculator`.
`mvn: command not found` or `docker: command not found` in the Jenkins log	Jenkins cannot find the tools. Add the `PATH+EXTRA` variable from step 3 of section 6.1.

Problem	Fix
Push fails with `denied: requested access to the resource is denied`	The credential ID must be `dockerhub-creds`, `DOCKERHUB_USER` must be your Docker Hub username, and the token needs Read & Write access.
A push does not start a build	Check that ngrok is still running and that the Payload URL ends with `/github-webhook/`. GitHub shows each delivery and its error under Settings > Webhooks > Recent Deliveries. Poll SCM is the fallback.
No email arrives	Check the spam folder, and make sure the SMTP credentials use a Gmail App Password.

9. Final test checklist

When you can tick all of these, every part of the project has been tested:

- [ ] `mvn test` reports 16 tests, 0 failures and BUILD SUCCESS.
- [ ] All 10 API checks in section 3 give the expected status and result.
- [ ] The image runs with one `docker run`, and `docker exec calc id` is not root.
- [ ] `docker compose ps` shows calculator **Up (healthy)** and calc-nginx **Up** on port 80, and the app's port 8080 is not published.
- [ ] The API and `/actuator/health` answer through Nginx on `http://localhost`.
- [ ] Jenkins shows six green stages and 16 passing tests, Docker Hub shows the new tag, and the success email arrives.
- [ ] A push to `main` starts the pipeline by itself, and the change is live on `http://localhost`.
- [ ] A broken test turns the pipeline red, nothing is deployed, and the failure email arrives.
- [ ] `./scripts/preflight.sh` ends with All preflight checks passed.